LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 06 – INHERITANCE



Maysa Azhra Fauziyyah

103132430005

S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

6.1 Pendahuluan

Inheritance (pewarisan) adalah salah satu pilar OOP, yaitu proses menurunkan suatu kelas dari kelas lain sehingga kelas turunannya mendapat ciri atau sifat dari kelas asalnya. Hubungannya disebut “is a relationship” (adalah bagian dari). Semakin ke bawah dalam hierarki, kelas semakin spesifik. Contohnya kelas Pria dan Wanita yang mewarisi sifat dari kelas Manusia. Tujuan utamanya adalah menghindari duplikasi field, variabel, maupun method. Sementara itu, polymorphism adalah kemampuan suatu objek untuk mengambil beberapa bentuk yang berbeda, yaitu satu interface yang sama bisa menghasilkan aksi atau proses yang berbeda.

6.2 Inheritance

Kelas yang menurunkan disebut superclass (base class), sedangkan kelas yang diturunkan disebut subclass (derived class). Di Java, pewarisan ditulis pakai keyword extends, misalnya class B extends A. Semua anggota superclass (atribut dan method) otomatis dimiliki subclass, dan subclass boleh menambah atribut atau method sendiri. Anggota yang private tidak bisa diakses langsung dari subclass.

6.3 Keyword Super

Keyword super dipakai untuk merujuk ke superclass dari suatu kelas. Ada tiga kegunaannya: super(...) untuk memanggil constructor superclass, super.atribut untuk mengakses atribut superclass, dan super.namaMethod() untuk memanggil method milik superclass.

6.4 Overloading Method

Overloading adalah membuat dua atau lebih method dengan nama yang sama dalam satu kelas, tapi jumlah atau tipe parameternya harus beda. Java akan memilih method yang dijalankan berdasarkan argumen yang dikirim saat dipanggil.

6.5 Overriding Method

Overriding terjadi kalau subclass mendefinisikan ulang method yang namanya dan parameternya sama dengan method di superclass. Kalau method itu dipanggil dari objek subclass, yang jalan adalah versi milik subclass. Versi superclass tetap bisa dipanggil lewat super.namaMethod(). Bedanya dengan overloading: overloading ada di satu kelas dengan parameter berbeda, sedangkan overriding ada di kelas turunan dengan parameter yang sama.

B. Guided

1. 6.2 Inheritance-Guided 1

#A
```
package com.maysa.projectmodul06.Guided.Gd1;

public class A {
    int x;
    int y;

    void tampilXY(){
        System.out.println("Nilai x = "+x+", y = "+y);
    }
}
```

#B
```
package com.maysa.projectmodul06.Guided.Gd1;

public class B extends A{
    int z;
    int hasil;

    void jumlahXYZ(){
        hasil = (x+y+z);

        System.out.println("Jumlah (x+y+z) = " + hasil);

    }
}
```

#Demo
```
package com.maysa.projectmodul06.Guided.Gd1;

public class Demo {
    public static void main(String[] args){

        A superclass = new A();
        B subclass = new B();

        superclass.x = 3;
        superclass.y = 4;

        System.out.println("Superclass: ");
        superclass.tampilXY();

        subclass.x = 1;
        subclass.y = 2;
        subclass.z = 5;

        System.out.println("Subclass: ");
        subclass.jumlahXYZ();
    }
}
```

Output :

```
----------------------< com.Maysa:projectmodul06 >----------------------
Building projectmodul06 1.0-SNAPSHOT
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.4.0:resources (default-resources) @ projectmodul06 ---
skip non existing resourceDirectory D:\SEMESTER 5\PEMROGRAMAN BERORIENTASI OBJECT\PR

--- compiler:3.15.0:compile (default-compile) @ projectmodul06 ---
Nothing to compile - all classes are up to date.

--- exec:3.6.3:exec (default-cli) @ projectmodul06 ---
Superclass:
Nilai x = 3, y = 4
Subclass:
Jumlah (x+y+z) = 8
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.646 s
Finished at: 2026-10-25T00:51:22+07:00
------------------------------------------------------------------------
```

Kesimpulan:

Kelas B yang extends A bisa langsung memakai atribut x dan y milik A tanpa mendeklarasikannya lagi, lalu menambah atribut z dan method jumlahXYZ() sendiri. Hasilnya 1 + 2 + 5 = 8. Ini menunjukkan subclass mewarisi anggota superclass sekaligus bisa punya anggota tambahan.

2. 6.3 Keyword Super-Guided 2

#Employee

```
package com.maysa.projectmodul06.Guided.Gd2;

public class Employee {
    private String name;
    String departemen;

    public Employee(String name){
        this.name = name;
    }

    public void tampilNama(){
        System.out.println("nama: "+ name);
    }
}
```

#Manager

```
package com.maysa.projectmodul06.Guided.Gd2;

public class Manager extends Employee {
    private String alamat;

    public Manager(String nama, String alamat){
        super(nama);
        this.alamat = alamat;
    }
```

```
    public void tampilInformasi(){
        super.tampilNama();
        System.out.println("alamat: " + alamat);
        System.out.println("departemen: " + super.departemen);
    }

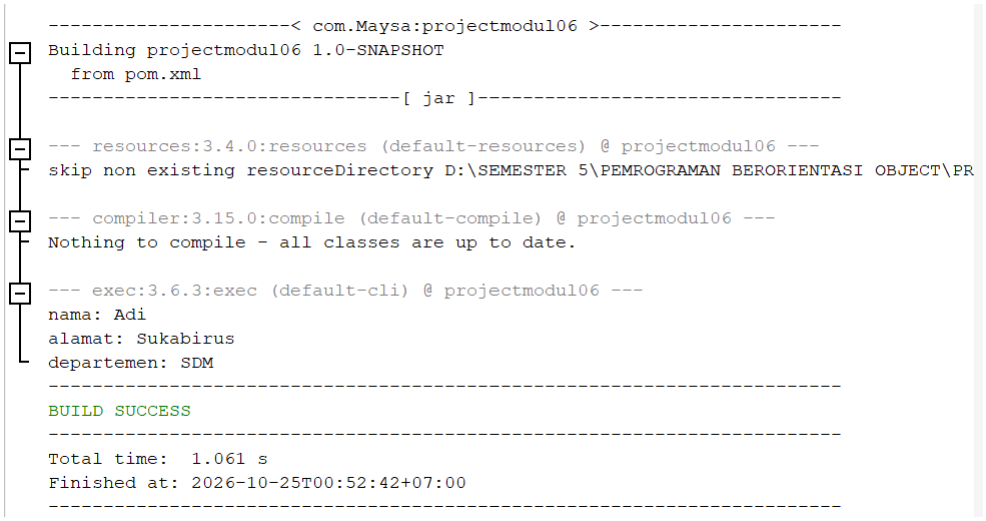
}
```

#Demo

```
package com.maysa.projectmodul06.Guided.Gd2;

public class Demo {
    public static void main(String[] args){
        Manager adi = new Manager("Adi", "Sukabirus");
        adi.departemen = "SDM";
        adi.tampilInformasi();
    }
}
```

Output:



```
----------------------< com.Maysa:projectmodul06 >----------------------
Building projectmodul06 1.0-SNAPSHOT
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.4.0:resources (default-resources) @ projectmodul06 ---
skip non existing resourceDirectory D:\SEMESTER 5\PEMROGRAMAN BERORIENTASI OBJECT\PR

--- compiler:3.15.0:compile (default-compile) @ projectmodul06 ---
Nothing to compile - all classes are up to date.

--- exec:3.6.3:exec (default-cli) @ projectmodul06 ---
nama: Adi
alamat: Sukabirus
departemen: SDM
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.061 s
Finished at: 2026-10-25T00:52:42+07:00
------------------------------------------------------------------------
```

Kesimpulan:

Constructor Manager memakai super(nama) untuk memanggil constructor Employee, karena atribut name bersifat private dan tidak bisa diisi langsung dari Manager. Method super.tampilNama() juga dipakai untuk memanggil method milik Employee, dan super.departemen untuk mengakses atribut superclass.

3. 6.4 Overloading Method-Guided 3

#Lagu

```
package com.maysa.projectmodul06.Guided.Gd3;

public class Lagu {
    String pencipta;
    String judul;

    void IsiParam(String param1){
```

```
        judul = param1;
        pencipta = "Tidak dikenal";
    }
    void IsiParam(String param1, String param2){
        judul = param1;
        pencipta = param2;
    }
    void tampilkan(){
        System.out.println("Judul: "+ judul + ", pencipta: " + pencipta);
    }
}
```

```
#Main
package com.maysa.projectmodul06.Guided.Gd3;

public class Main {
    public static void main(String[] args){
        Lagu d,e;
        d = new Lagu();
        e = new Lagu();

        d.IsiParam("Lagu 1");
        e.IsiParam("Lagu 2", "aku");

        d.tampilkan();
        e.tampilkan();
    }
}
```

Output:

```
----------------------< com.Maysa:projectmodul06 >----------------------
Building projectmodul06 1.0-SNAPSHOT
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.4.0:resources (default-resources) @ projectmodul06 ---
skip non existing resourceDirectory D:\SEMESTER 5\PEMROGRAMAN BERORIENTASI OBJECT\PR

--- compiler:3.15.0:compile (default-compile) @ projectmodul06 ---
Nothing to compile - all classes are up to date.

--- exec:3.6.3:exec (default-cli) @ projectmodul06 ---
Judul: Lagu 1, pencipta: Tidak dikenal
Judul: Lagu 2, pencipta: aku
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.151 s
Finished at: 2026-10-25T00:53:24+07:00
------------------------------------------------------------------------
```

Kesimpulan:

Method IsiParam() dibuat dua versi dengan jumlah parameter berbeda. Saat dipanggil dengan 1 argumen (d), versi pertama jalan dan pencipta otomatis terisi “Tidak dikenal”. Saat dipanggil dengan 2 argumen (e), versi kedua jalan dan pencipta terisi sesuai argumen. Java memilih versi yang cocok dengan jumlah argumennya.

4. 6.5 Overriding Method-Guided 4

```
#A
package com.maysa.projectmodul06.Guided.Gd4;

public class A {
    public void tampilkan() {
        System.out.println("Method milik class A dipanggil...");
    }
}
```

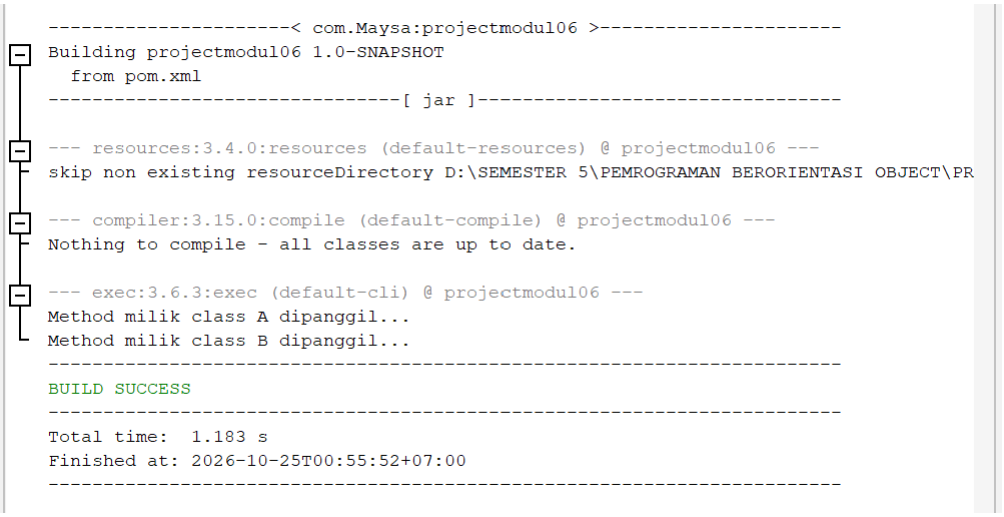
```
#B
package com.maysa.projectmodul06.Guided.Gd4;

public class B extends A {
    public void tampilkan() {
        super.tampilkan();
        System.out.println("Method milik class B dipanggil...");
    }
}
```

```
#Main
package com.maysa.projectmodul06.Guided.Gd4;

public class Main {
    public static void main(String[] args) {
        B overidB = new B();
        overidB.tampilkan();
    }
}
```

Output:



Kesimpulan:

Kelas B meng-override method tampilkan() milik A. Saat dipanggil dari objek B, yang jalan adalah versi B. Tapi karena di dalamnya ada super.tampilkan(), versi A ikut dijalankan lebih dulu, makanya output-nya muncul dua baris.

C. Unguided

Soal Unguided — Modul 6 Inheritance

Sebuah tim *machine learning* ingin membuat program untuk membandingkan estimasi biaya komputasi dari beberapa jenis model yang akan dilatih. Ada tiga jenis model: model regresi, model klasifikasi, dan model klasifikasi berbasis *deep learning* yang merupakan pengembangan dari model klasifikasi biasa.

Buatlah program Java dengan ketentuan: Penamaan kelas, atribut, dan method ditentukan sendiri oleh Anda dan semua atribut bersifat private.

Ketentuan

1. Kelas induk (model umum)

- Menyimpan **nama model** (teks) dan **jumlah data latih** (bilangan bulat).
- Memiliki *constructor* yang menerima kedua nilai tersebut, serta cara bagi kelas lain untuk membaca jumlah data latih.
- Memiliki method untuk menampilkan informasi.
- Memiliki dua method untuk menghitung biaya komputasi dengan nama yang sama (*overloading*):
    - Tanpa parameter: biaya = jumlah data latih × 2.
    - Dengan satu parameter bilangan bulat / jumlah epoch: biaya = jumlah data latih × epoch.

2. Kelas turunan pertama (model regresi)

- Mewarisi kelas induk dan tambahkan atribut **jumlah fitur**.
- *Constructor* menerima nama, jumlah data latih, dan jumlah fitur; gunakan *constructor* kelas induk.
- **Override** method: tampilkan informasi dari kelas induk.
- **Override** method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) + (jumlah fitur × 10).
- Tambahkan **satu method baru** dengan nama yang sama dengan method penghitung biaya, tetapi menerima **dua parameter bilangan bulat** (jumlah *epoch* dan ukuran *batch*): biaya = jumlah data latih × epoch ÷ ukuran batch. Sehingga objek kelas ini memiliki **tiga versi** method penghitung biaya.

3. Kelas turunan kedua (model klasifikasi)

- Mewarisi kelas induk dan menambahkan atribut **jumlah kelas**.
- *Constructor* menerima nama, jumlah data latih, dan jumlah kelas; gunakan *constructor* kelas induk.
- **Override** method: tampilkan informasi dari kelas induk.
- **Override** method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) × jumlah kelas.
- Versi penghitung biaya dengan parameter *epoch* **tidak** di-override.

4. Kelas turunan ketiga (model klasifikasi *deep learning*)

- Mewarisi **kelas turunan kedua** (bukan kelas induk langsung) dan menambahkan atribut **jumlah layer**.
- *Constructor* menerima nama, jumlah data latih, jumlah kelas, dan jumlah layer; gunakan *constructor* kelas di atasnya.
- **Override** method: tampilkan informasi dari kelas di atasnya.
- **Override** method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas di atasnya) + (jumlah layer × 200).

5. Class Main

- Buat tiga objek berikut:

Jenis model	Nama	Data latih	Atribut tambahan
Regresi	"Regresi Linier"	500	4 fitur
Klasifikasi	"Decision Tree"	800	3 kelas
Klasifikasi *deep learning*	"Neural Network"	1000	5 kelas, 3 layer

- Simpan ketiga objek tersebut dalam satu array bertipe kelas induk.
- Gunakan perulangan untuk setiap elemen array: tampilkan informasi model, tampilkan biaya komputasi, lalu jumlahkan seluruh biaya.
- Setelah perulangan, tampilkan total biaya.
- Terakhir, tampilkan juga:
    - biaya 10 *epoch* untuk model "Neural Network" (ambil dari array),
    - biaya 10 *epoch* untuk model "Regresi Linier",
    - biaya 10 *epoch* dengan ukuran *batch* 50 untuk model "Regresi Linier".

Contoh Output

```
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
```

```
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
```

Jawaban :

Package (model & main)

1. Unguided-model-ModelML

```
package com.maysa.projectmodul06.Unguided.model;

//Kelas induk: model machine learning umum
public class ModelML {
    private String namaModel;
    private int jumlahDataLatih;

    public ModelML(String namaModel, int jumlahDataLatih) {
        this.namaModel = namaModel;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    // Getter agar kelas turunan/kelas lain dapat membaca jumlah data latih
    public int getJumlahDataLatih() {
        return jumlahDataLatih;
    }

    public void tampilkanInfo() {
        System.out.println("Model: " + namaModel + " | Data latih: " + jumlahDataLatih);
    }

    // Overloading 1: tanpa parameter
    public int hitungBiaya() {
        return jumlahDataLatih * 2;
    }

    // Overloading 2: satu parameter (epoch)
    public int hitungBiaya(int epoch) {
        return jumlahDataLatih * epoch;
    }
}
```

2. Unguided-model-ModelRegresi

```
package com.maysa.projectmodul06.Unguided.model;

//Kelas turunan pertama: model regresi
public class ModelRegresi extends ModelML {
    private int jumlahFitur;

    public ModelRegresi(String namaModel, int jumlahDataLatih, int jumlahFitur) {
        super(namaModel, jumlahDataLatih);
```

```
        this.jumlahFitur = jumlahFitur;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    // Override: biaya induk + (fitur x 10)
    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    // Overloading tambahan: dua parameter (epoch dan ukuran batch)
    public int hitungBiaya(int epoch, int ukuranBatch) {
        return getJumlahDataLatih() * epoch / ukuranBatch;
    }
}
```

3. Unguided-model-ModelKlasifikasi

```
package com.maysa.projectmodul06.Unguided.model;

// Kelas turunan kedua: model klasifikasi
public class ModelKlasifikasi extends ModelML {
    private int jumlahKelas;

    public ModelKlasifikasi(String namaModel, int jumlahDataLatih, int jumlahKelas) {
        super(namaModel, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }

    // Override: biaya induk x jumlah kelas
    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() * jumlahKelas;
    }

    // hitungBiaya(int epoch) TIDAK di-override (memakai versi kelas induk)
}
```

4. Unguided-model-ModelDeepLearning

```
package com.maysa.projectmodul06.Unguided.model;
```

```
// Kelas turunan ketiga: mewarisi ModelKlasifikasi (bukan ModelML langsung)
public class ModelDeepLearning extends ModelKlasifikasi {
    private int jumlahLayer;

    public ModelDeepLearning(String namaModel, int jumlahDataLatih, int jumlahKelas,
int jumlahLayer) {
        super(namaModel, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah layer: " + jumlahLayer);
    }

    // Override: biaya ModelKlasifikasi + (layer x 200)
    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahLayer * 200);
    }
}
```

5. Unguided-main-Main

```
package com.maysa.projectmodul06.Unguided.main;

import com.maysa.projectmodul06.Unguided.model.ModelML;
import com.maysa.projectmodul06.Unguided.model.ModelRegresi;
import com.maysa.projectmodul06.Unguided.model.ModelKlasifikasi;
import com.maysa.projectmodul06.Unguided.model.ModelDeepLearning;

public class Main {
    public static void main(String[] args) {
        // Array bertipe kelas induk (polymorphism)
        ModelML[] daftarModel = {
            new ModelRegresi("Regresi Linier", 500, 4),
            new ModelKlasifikasi("Decision Tree", 800, 3),
            new ModelDeepLearning("Neural Network", 1000, 5, 3)
        };

        int totalBiaya = 0;
        for (ModelML model : daftarModel) {
            model.tampilkanInfo();
            int biaya = model.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");
            totalBiaya += biaya;
        }
        System.out.println("Total biaya: " + totalBiaya);
```

```
        // Neural Network (indeks 2): hitungBiaya(int) milik ModelML, tidak di-
override
        System.out.println("Biaya 10 epoch (Neural Network): " +
daftarModel[2].hitungBiaya(10));

        // Regresi Linier (indeks 0)
        System.out.println("Biaya 10 epoch (Regresi Linier): " +
daftarModel[0].hitungBiaya(10));

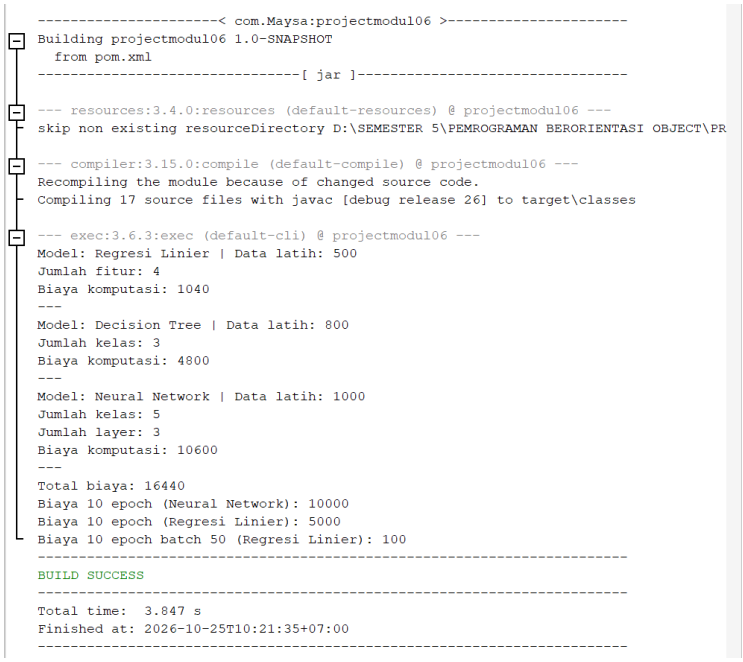
        // Versi dua parameter hanya ada di ModelRegresi, sehingga perlu casting
        ModelRegresi regresi = (ModelRegresi) daftarModel[0];
        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): " +
regresi.hitungBiaya(10, 50));
    }
}
```

Penjelasan source code:

Program ini membandingkan estimasi biaya komputasi dari tiga jenis model machine learning, dengan struktur pewarisan bertingkat: ModelML → ModelRegresi, dan ModelML → ModelKlasifikasi → ModelDeepLearning. Semua atribut dibuat private (enkapsulasi) dan constructor memakai this.

- **ModelML (kelas induk).** Menyimpan namaModel dan jumlahDataLatih, lengkap dengan constructor, getter getJumlahDataLatih() supaya kelas turunan bisa membaca datanya, dan method tampilkanInfo(). Ada dua method hitungBiaya() yang saling overloading: tanpa parameter (data latih × 2) dan dengan parameter epoch (data latih × epoch).
- **ModelRegresi.** Mewarisi ModelML dan menambah jumlahFitur. Constructor-nya memanggil super(...). tampilkanInfo() dan hitungBiaya() di-override, dan biayanya jadi biaya induk + (fitur × 10). Ada tambahan overloading hitungBiaya(int epoch, int ukuranBatch) = data latih × epoch ÷ ukuran batch, jadi objek kelas ini punya tiga versi hitungBiaya().
- **ModelKlasifikasi.** Mewarisi ModelML dan menambah jumlahKelas. tampilkanInfo() dan hitungBiaya() di-override, dengan biaya = biaya induk × jumlah kelas. Versi hitungBiaya(int epoch) sengaja tidak di-override, jadi tetap memakai versi ModelML.
- **ModelDeepLearning.** Mewarisi ModelKlasifikasi (bukan ModelML langsung) dan menambah jumlahLayer. Biayanya = biaya ModelKlasifikasi + (layer × 200), dengan super yang merujuk ke kelas di atasnya.
- **Main.** Tiga objek disimpan dalam satu array bertipe ModelML (polimorfisme). Perulangan memanggil tampilkanInfo() dan hitungBiaya() untuk tiap elemen, lalu menjumlahkan biayanya. Setelah itu ditampilkan total biaya dan biaya 10 epoch. Method dua parameter hanya ada di ModelRegresi, jadi elemen array di-cast dulu ke ModelRegresi sebelum dipanggil.

Output:



```
----------------------< com.Maysa:projectmodul06 >----------------------
Building projectmodul06 1.0-SNAPSHOT
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.4.0:resources (default-resources) @ projectmodul06 ---
skip non existing resourceDirectory D:\SEMESTER 5\PEMROGRAMAN BERORIENTASI OBJECT\PR

--- compiler:3.15.0:compile (default-compile) @ projectmodul06 ---
Recompiling the module because of changed source code.
Compiling 17 source files with javac [debug release 26] to target\classes

--- exec:3.6.3:exec (default-cli) @ projectmodul06 ---
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  3.847 s
Finished at: 2026-10-25T10:21:35+07:00
------------------------------------------------------------------------
```

Penjelasan Output:

Walaupun ketiga objek disimpan di array bertipe ModelML, tiap pemanggilan tampilkanInfo() dan hitungBiaya() tetap menjalankan versi milik kelas aslinya (polimorfisme), makanya info yang tampil beda-beda. Perhitungannya:

- Regresi Linier: 500 × 2 = 1000, lalu + (4 × 10) = 1040.
- Decision Tree: 800 × 2 = 1600, lalu × 3 kelas = 4800.
- Neural Network: ModelML 1000 × 2 = 2000, ModelKlasifikasi × 5 = 10000, lalu ModelDeepLearning + (3 × 200) = 10600.
- Total biaya: 1040 + 4800 + 10600 = 16440.

Untuk 10 epoch, Neural Network dan Regresi Linier sama-sama memakai versi ModelML karena hitungBiaya(int epoch) tidak di-override, jadi 1000 × 10 = 10000 dan 500 × 10 = 5000. Terakhir, versi dua parameter milik ModelRegresi menghasilkan 500 × 10 ÷ 50 = 100.

D. Kesimpulan

Dari bagian guided, empat konsep dasar pewarisan dipraktikkan lewat contoh terpisah. Pada inheritance (Guided 1), subclass B memakai atribut x dan y dari A tanpa mendeklarasikannya ulang, lalu menambah atribut dan method sendiri. Pada keyword super (Guided 2), Manager memakai super(...) untuk memanggil constructor Employee, serta super.tampilNama() dan super.departemen untuk mengakses anggota superclass. Pada overloading (Guided 3), method IsiParam() punya dua versi dengan jumlah parameter berbeda dan Java memilih versi yang sesuai dengan argumennya. Pada overriding (Guided 4), B mendefinisikan ulang tampilkan() milik A sehingga versi B yang jalan, dengan super.tampilkan() untuk tetap memanggil versi A.

Pada bagian unguided, semua konsep itu digabung dalam program estimasi biaya komputasi model machine learning. Ada pewarisan bertingkat (ModelML → ModelKlasifikasi → ModelDeepLearning), super untuk memanggil constructor dan method kelas di atasnya, overloading hitungBiaya() dengan tiga versi parameter, serta overriding tampilkanInfo() dan hitungBiaya() di kelas turunan. Ketiga objek disimpan dalam satu array bertipe kelas induk dan tetap menjalankan versi method milik kelas aslinya masing-masing, sehingga hasilnya sesuai: total biaya 16440, biaya 10 epoch 10000 dan 5000, dan biaya dengan batch 50 sebesar 100.

Dari keseluruhan praktikum, dapat disimpulkan bahwa inheritance mengurangi duplikasi kode karena kelas turunan tinggal memakai dan menambah apa yang sudah ada di kelas induknya. Overloading dipakai untuk satu nama method dengan parameter berbeda dalam satu kelas, sedangkan overriding dipakai untuk mengubah perilaku method induk di kelas turunan. Dengan begitu, tujuan praktikum untuk memahami inheritance dan polimorfisme serta menerapkannya dalam program Java sudah tercapai.